COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/02/26	**DATE SUBMITTED:**	10/06/26
NAME:	Garcia, Dennis James T. Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY 10.1 GRAPHS

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

Source Code

```cpp
#include <iostream>
// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
// insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head){
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head; // point new node to current head
    return newNode;
    }

    int N; // number of nodes in the graph
```

```cpp
    public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N){
    // allocate new node
        head = new adjNode*[N]();
        this->N = N;
    // initialize head pointer for all vertices
    for (int i = 0; i < N; ++i)
        head[i] = nullptr;
    // construct directed graph by adding edges to it
    for (unsigned i = 0; i < n; i++){
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;
    // insert in the beginning
        adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
    // point head pointer to new node
        head[start_ver] = newNode;
        }
    }
```

Output:

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Source Code

main.cpp > Graph<T>

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>

class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }

    return os;
}
```

```cpp
template <typename T>
class Graph {

public:

    Graph(size_t N) : V(N) {}

    auto vertices()const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if  (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

```cpp
template <typename T>
void test_DFS()
{

    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\Hare\CPE010_HOA10p1_HareGarcia> & C:\Users\TIPQC\Documents\Hare\CPE010_HOA10p1_HareGarcia\main.exe
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

Source Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>

class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }
inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){

        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

    os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph{

public:
    Graph(size_t N) : V(N) {}

    auto vertices() const{
    return V;
}
    auto &edges() const{
    return edge_list;
}

void add_edge(Edge<T> &&e){
    if (e.src >= 1 && e.src <= V &&
        e.dest >= 1 && e.dest <= V)
        edge_list.emplace_back(e);
    else
        std::cerr << "Vertex out of bounds" << std::endl;
}
```

```
auto outgoing_edges(size_t v) const{
    std::vector<Edge<T>> edges_from_v;
    for (auto &e : edge_list){
        if (e.src == v)
            edges_from_v.emplace_back(e);
    }
    return edges_from_v;
}

template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```
template <typename T>
auto create_reference_graph(){
Graph<T> G(9);
std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
for (auto &i : edges){
    for (auto &j : i.second){
        G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
return G;
}
```

```
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest){
std::queue<size_t> queue;
std::vector<size_t> visit_order;
std::set<size_t> visited;
queue.push(1); // Assume that BFS always starts from vertex ID 1

while (!queue.empty()){
    auto current_vertex = queue.front();
    queue.pop();
// If the current vertex hasn't been visited in the past
if (visited.find(current_vertex) == visited.end()){
    visited.insert(current_vertex);
    visit_order.push_back(current_vertex);
        for (auto e : G.outgoing_edges(current_vertex)){
            queue.push(e.dest);
            }
        }
    }
    return visit_order;
}
```

```
template <typename T>
void test_BFS(){
// Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
// Run BFS starting from vertex ID 1 and print the order
// in which vertices are visited.
        std::cout << "BFS Order of vertices: " << std::endl;
auto bfs_visit_order = breadth_first_search(G, 1);
for (auto v : bfs_visit_order)
    std::cout << v << std::endl;

}
```

Output

```
PS C:\Users\TIPQC\Documents\CPE 010_Garcia_CPE21S4\HoA 10.1>
rs\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-
 '--pid=Microsoft-MIEngine-Pid-0wc1wcti.fcg' '--dbgExe=C:\msy
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
```

```
BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\Documents\CPE 010_Garcia
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

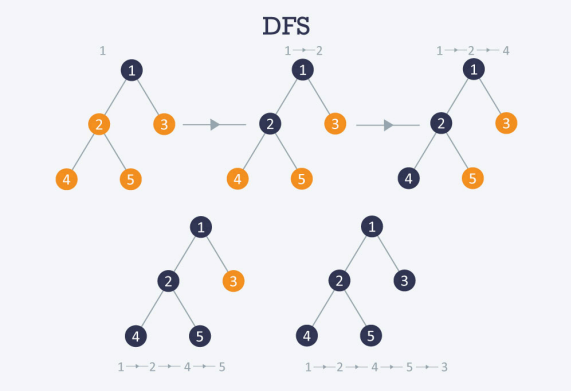
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

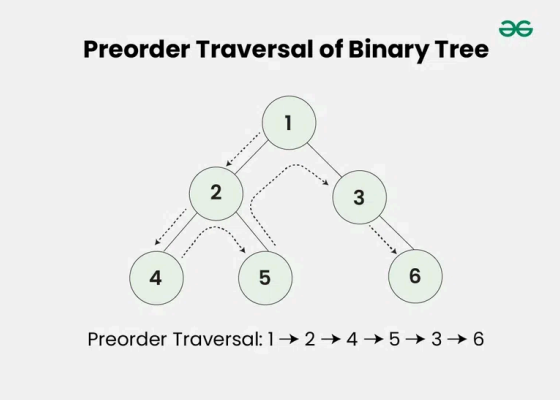
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The Undirected Graph shows that you can visit any vertex and seems to be backtrackable, basing this on the ILOA of the discussion. Looking at the adjacency list, you can go from A to B then to C then to D; backtracking to C then to A and ending with D.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

Graphical comparison





Pseudocode

```
function preorder(node):
  //Input: root node of tree
  //Output: None
  visit(node)
  if node has left child
    preorder(node.left)
  if node has right child
    preorder(node.right)
```

Code Implementation

```
template <typename T>
void BinarySearchTree<T>::insert(T value){
    if(value < key){
        //do something
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }else{
            left->insert(value);
        }

    }else if (value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right->insert(value);
        }
    }
}
```

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<<key<<" ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

The equivalence of the DFS is the preorder traversal since they traverse the left child node first before going to the right child node. In the graphs, it goes through the bottom of the tree on its left side then moves to its right child node if there is one available; then it traverses to the right side of the tree going through the very bottom of it again.

3. In the performed code, what data structure is used to implement the Breadth First Search?

Based on the code that we performed, we observed that a queue is used for BFS. We can see the std::queue<size_t> queue in the code and the nodes are added and removed from this queue while the graph is being traversed.It allows BFS to visit the nodes starting from the first node and then continue to the next connected nodes.

4. How many times can a node be visited in the BFS?

Upon observing, we see that a node is visited only once. It uses a code set to keep track of nodes that have already been visited so before adding a node to the visit order, the code checks first if it has already been visited.

Conclusion	
Summary of lessons learned	- Hare I learned the basic concepts of graphs and how nodes can be connected and visited. I also learned that BFS and DFS use different ways of traversing a graph. - Garcia I learned the similarity of some search trees that were done in previous HoAs based on their concepts and the difference of implementation in this HoA as well.
Analysis of the procedure	- Hare I observed how the graph was created and how the vertices and edges were connected then also on how DFS and BFS were used to traverse the graph. - Garcia I observed that the graph doesn't really have a root to start but it has traversal functions that were similar to preorder from the binary search tree
Analysis of the supplementary activity	- Hare It helped me understand the purpose of the data structures used in graph traversal. I see that BFS uses a queue and keeps track of visited nodes that helps the program process the graph without repeatedly visiting the same node. - Garcia The answer I provided made me understand the difference in implementation and concepts (with the absence of root node) while the graphs similarities to search trees and its search traversal.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	- Hare I think I understood the general idea of how graphs and graph traversal work after doing the activity. I still need more practice in tracing the nodes and understanding the C++ code of this topic.. - Garcia I somewhat understood the concepts or general idea but the implementation has many elements to comprehend. I may need to familiarize myself with vertices and edges.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.